COMP 523 · TEAM A · DELIVERABLE D2

Nora Design Document

A tutorial for the code behind Nora the Policy Navigator, written for a developer who needs to maintain it, extend it, or reuse parts of it.

Andy Huoy · Adithi Srikrishna · Anton Sun · Jerry Wen · Last updated Nov 2, 2026
Describes the `dev` branch of github.com/jerrywen2005/Nora-Research-Policy-Navigator at commit `60fa742`

How to use this document

It assumes you know basic programming and object-oriented ideas, and nothing about UNC research policy. Domain terms (Onyen, RAG, CloudApps…) are explained in the Glossary.

- **New to the project?** Read sections 1 and 2, then run it with the README.
- **Have a specific task?** Start at section 5, which links back to what you need.
- **Touching the database?** Section 3 defines every table and field.
- **Wondering why it's built this way?** Section 4.

Nora is partly built. Built parts exist and work, Mockup parts show sample data, and Planned parts are designed but not written yet.

Contents

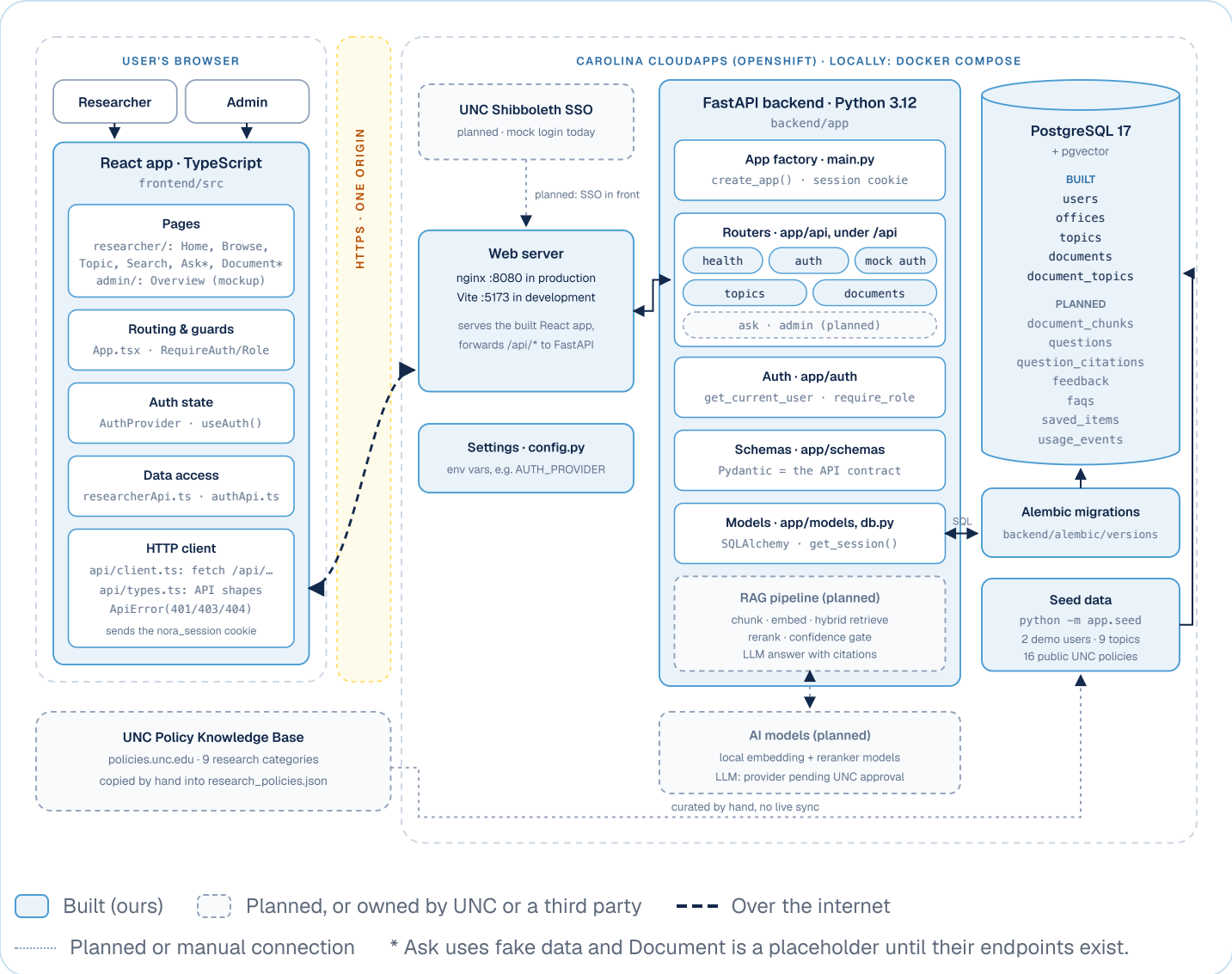
1. Architecture
2. Code Repository
3. Detailed Data Definitions
4. Design Rationale
5. Guide for the Next Developer
6. Related Documents
7. Glossary

What Nora does, in one paragraph

UNC research policy is spread across many websites and offices, and it's hard to interpret even once you find it. Nora gives researchers one place to **browse** policy by topic, **search** it by keyword, or **ask** a plain-language question and get a short answer that cites the policy it came from, or a referral to the right office when the policies don't clearly answer it. UNC Research staff (admins) manage the content and see what researchers ask. Our client is UNC's Office of the Vice Chancellor for Research.

1. Architecture

Nora is a three-tier, client-server web application: a React app in the browser, a FastAPI server, and a PostgreSQL database. It works by request and response; nothing is event-driven or real-time. This diagram expands our first architecture diagram with the modules that now exist.



The parts

React app (frontend/) Built

Runs in the user's browser. Sign-in, the researcher pages (home, Browse, Topic, Search) and the routing and role guards are built and use the real API. Ask uses fake data and the document page is a placeholder until their endpoints exist. The admin Overview is a mockup with sample numbers.

Web server (nginx or Vite) Built

Serves the React app and forwards every `/api/*` request to FastAPI, so the browser only talks to one origin and no CORS setup is needed. Vite does this in development and nginx in production.

FastAPI backend (backend/app/) Built

All application logic, as a JSON API under `/api`: sign-in and roles, topics, and document browse and search. The Ask endpoint (RAG) and the admin endpoints are planned.

PostgreSQL 17 + pgvector Built

The only data store. Five tables exist (users, offices, topics, documents, document_topics); the RAG and analytics tables are designed but not yet created.

UNC Shibboleth SSO Planned

UNC's Onyen login. Access has been requested. Until then, a mock login provider models it, and switching is a configuration change plus one new provider module.

AI models Planned

Local embedding and reranker models to start (no researcher data leaves our servers), and an LLM whose provider UNC hasn't approved yet. Both sit behind interfaces so they can be swapped and faked in tests.

Following one request: a keyword search

This trace shows how the layers in the diagram connect. Every other feature follows the same path, with a different page, router and query.

1 **`frontend/src/pages/researcher/SearchPage.tsx`**

The researcher types "irb exemption". The page keeps the query in the URL (`/search?q=irb+exemption`) and calls `getDocuments({ q })`.

2 **`frontend/src/pages/researcher/researcherApi.ts`**

`getDocuments` builds the query string and calls `apiGet('/documents?q=irb+exemption')`. Pages never call `fetch` themselves.

3 **`frontend/src/api/client.ts`**

`apiGet` fetches the relative path `/api/documents?...`. The browser sends the `nora_session` cookie automatically. A non-2xx response becomes an `ApiError` carrying the status.

4 **`frontend/vite.config.ts` · `frontend/nginx.conf.template`**

The Vite dev proxy (or nginx in production) forwards `/api/*` to FastAPI on port 8000.

5 **`backend/app/main.py`**

`SessionMiddleware` verifies the cookie's signature and exposes its contents as `request.session`. The request reaches the documents router.

6 **`backend/app/api/documents.py` · `backend/app/auth/dependencies.py`**

The router's `Depends(get_current_user)` reads the user id from the session and loads the user, returning 401 if there is none or the account is disabled. `list_documents` validates the query parameters, then calls `search_documents()`.

7 **`backend/app/api/documents.py` · `backend/app/models/content.py`**

`search_documents()` builds a SQLAlchemy query: active documents whose `search_vector` matches `websearch_to_tsquery('english', q)`, ranked by `ts_rank_cd`, with each document's office and topics loaded.

8 **`PostgreSQL`**

The GIN index on `documents.search_vector` finds the matches. Title matches outrank summary matches, which outrank body matches.

9 **`backend/app/schemas/content.py` → `frontend/src/api/types.ts`**

FastAPI serializes the rows through the `DocumentPage` Pydantic schema as `{ items, total }`, matching the TypeScript `Page<DocumentSummary>` type. The page renders each result with its citation, office and source link.

How sign-in works

1. On load, `AuthProvider` calls `GET /api/me`. A 401 means signed out, so `RequireAuth` sends the user to `/signin`, remembering the page they wanted.
2. The sign-in page posts an Onyen to `POST /api/auth/mock-login`. The backend trims and lowercases it, checks it against `[a-z0-9._-]{1,64}`, and calls `record_login()`, which finds or creates the user (new users are researchers) and stamps `last_login_at`.
3. The backend stores only the user's id in the signed `nora_session` cookie (HTTP-only, SameSite=Lax, 8 hours, HTTPS-only in production) and returns the user.
4. The frontend sends admins to `/admin` and researchers to `/`. `RequireRole` hides the admin area from researchers, but the backend's `require_role()` is what actually enforces it, returning 403.
5. Every later request re-reads the user and role from the database, so disabling a user or changing their role takes effect immediately.

Local vs. production

	LOCAL (DOCKER COMPOSE)	PRODUCTION (CAROLINA CLOUDAPPS)
Serves the React app	Vite dev server, port 5173, hot reload	nginx-unprivileged, port 8080
Forwards /api/*	Vite's `server.proxy`	nginx `proxy_pass` to `BACKEND_URL`
API	uvicorn with `--reload`; migrates and seeds on start	uvicorn as a non-root user; run `alembic upgrade head` before rollout
Database	`pgvector/pgvector:pg17` container	PostgreSQL + pgvector, set up by the client's team
Sign-in	Mock login (`AUTH_PROVIDER=mock`)	UNC Shibboleth SSO; the app refuses to start with mock auth
Data	Seed data: 2 fake users, 9 topics, 16 public policies	Client-approved documents and real users

2. Code Repository

Nora's code is on GitHub at github.com/jerrywen2005/Nora-Research-Policy-Navigator. Work happens on the `dev` branch (the default); `main` is reserved for production releases.

```
git clone https://github.com/jerrywen2005/Nora-Research-Policy-Navigator.git
cd Nora-Research-Policy-Navigator
docker compose up --build     # then open http://localhost:5173
```

Languages, and which parts use them

LANGUAGE	WHERE	DIAGRAM PARTS WRITTEN IN IT
Python 3.12	`backend/`	The FastAPI backend: routers, auth, schemas, models, migrations, seed script and tests.
TypeScript (TSX)	`frontend/src/`	The React app: pages, components, routing, auth state and API client.
CSS	`frontend/src/**/*.css`	Styles, as plain CSS files next to the components that use them, with shared variables in `theme.css`.
SQL	inside the Python code	Written mostly through SQLAlchemy. Raw SQL appears in the generated `search_vector` expression and the CHECK constraints.
JSON	`backend/app/seed/data/`	The 16 seed policies with their full text.
YAML, Dockerfile, nginx config	repo root, `backend/`, `frontend/`, `.github/`	Docker Compose, the container images, the production web server and the CI workflows.
Markdown	`README.md`, `CLAUDE.md`, `docs/`	Setup instructions and the project's design records.

Repository layout

```
Nora-Research-Policy-Navigator/
├── README.md              setup, stack, workflow (copied below)
├── CLAUDE.md              short project rules for contributors and AI assistants
├── docker-compose.yml     local stack: db, backend, frontend
├── .env.example           environment variables for local runs
├── .github/workflows/     ci.yml (lint, tests, migrations, image builds), branch-rules.yml
├── docs/                  design records: product, API, database, auth, RAG, decisions…
```

```
├── backend/              FastAPI app (Python)
│   ├── app/
│   │   ├── main.py       create_app(): middleware and routers
│   │   ├── config.py     settings from environment variables
│   │   ├── db.py         engine, Base, get_session()
│   │   ├── api/          routers: health, auth, topics, documents
│   │   ├── auth/         get_current_user, require_role, record_login
│   │   ├── models/       SQLAlchemy tables: user.py, content.py
│   │   ├── schemas/      Pydantic request/response shapes
│   │   └── seed/         python -m app.seed, plus data/research_policies.json
│   ├── alembic/versions/ database migrations
│   └── tests/            pytest suite (100% coverage required)
└── frontend/             React app (TypeScript, Vite)
    ├── nginx.conf.template production web server config
    └── src/
        ├── main.tsx, App.tsx  entry point and every route
        ├── api/          client.ts (fetch wrapper), types.ts (API shapes)
        ├── auth/         AuthProvider, useAuth, RequireAuth, RequireRole
        ├── layout/       AppLayout, header, sidebar, navItems.ts
        ├── pages/        SignIn, researcher/, admin/
        └── components/   shared pieces for the researcher pages
```

Backend modules (Python)

Listed from the outside in, matching the FastAPI box in the diagram. Paths are relative to `backend/`.

MODULE	WHAT IT DOES	USES
`app/main.py` Built	`create_app()` builds the FastAPI app: session middleware, optional CORS, and every router under `/api`. The mock login router is mounted only when `AUTH_PROVIDER=mock`.	config, api/*
`app/config.py` Built	`Settings` reads environment variables (and a local `.env`). Refuses to start in production with mock auth or the development session secret.	nothing; used by everything
`app/db.py` Built	The SQLAlchemy engine, the `Base` class every model extends (with a constraint naming convention), and the `get_session()` dependency.	config
`app/api/` Built	One router per area: `health.py`, `auth.py` (`/me`, `/auth/logout`, `/auth/mock-login`), `topics.py` and `documents.py` (which also holds `search_documents()`, the future RAG keyword retriever).	auth, models, schemas, db

MODULE	WHAT IT DOES	USES
app/auth/ Built	`get_current_user` (session → user, or 401), `require_role(role)` (or 403) and `record_login()` (find or create the user). Routes depend on these and never read cookies or headers themselves, so the auth provider can be swapped.	models, db
app/models/ Built	The database tables as SQLAlchemy classes: `user.py` (`User`, `Role`) and `content.py` (`Office`, `Topic`, `Document`, `document_topics`, `DocType`, `DocumentStatus`). Every model must be imported in `__init__.py` so Alembic sees it.	db
app/schemas/ Built	Pydantic models for request and response bodies: `user.py` (`UserOut`, `MockLoginRequest` with Onyen validation) and `content.py`. They must match `docs/api.md` and `frontend/src/api/types.ts` exactly.	nothing
app/seed/ Built	`python -m app.seed` runs idempotent `seed_users` and `seed_content`, loading the fake users, 8 offices, 9 topics and the policies in `data/research_policies.json`.	models, db
alembic/ Built	Migrations, applied in order: enable pgvector (`7fee7dd7b6ac`) → users (`6ddc0a734e48`) → content tables (`6de3ee5b4d8c`).	models (for autogenerate)
tests/ Built	pytest against a real Postgres. Each test runs in a transaction that is rolled back. `conftest.py` has the `db_session`, `client` and `make_user` fixtures and a `login(client, onyen)` helper.	everything
app/api/ask.py, app/rag/ (names not final) Planned	`POST /api/ask`: retrieve, rerank, confidence gate, LLM answer with citations, then log the question. Design in `docs/rag.md`.	models, AI providers
app/api/admin/ (name not final) Planned	Admin endpoints under `/api/admin/*` for the admin list, content management and analytics, each guarded by `require_role(Role.ADMIN)`.	auth, models

Frontend modules (TypeScript)

Paths are relative to `frontend/`.

MODULE	WHAT IT DOES	USES
`src/main.tsx`, `src/App.tsx` Built	The entry point (router and global CSS) and every route. All routes except `/signin` sit under `RequireAuth`; `/admin` also sits under `RequireRole role="admin"`.	auth, layout, pages
`src/api/` Built	`client.ts`: `apiGet` and `apiPost` over `fetch`, throwing `ApiError` with the status. `types.ts`: TypeScript versions of every API shape.	nothing
`src/auth/` Built	`AuthProvider` and `useAuth()` (the current user, `signIn`, `signOut`), the `RequireAuth` and `RequireRole` route guards, and `authApi.ts`.	api
`src/layout/` Built	`AppLayout` (header, sidebar and page), `UserMenu`, `NotFound`, and `navItems.ts`. A nav item without a `to` shows disabled with a "Soon" tag.	auth
`src/pages/SignIn.tsx` Built	The demo sign-in page, with an Onyen field and one-click researcher and admin demo buttons. It's deliberately styled as Nora, not as UNC's real login page.	auth
`src/pages/researcher/` Built	`HomePage`, `BrowsePage`, `TopicPage`, `SearchPage` (real API); `AskPage` (fake data until `POST /api/ask`); `DocumentPage` (placeholder). All data comes through `researcherApi.ts`.	api, components
`src/components/` Built	Shared pieces for the researcher pages: `TopicGrid`, `SearchBar`, `DocumentResultCard`, `AskPanel`, `AnswerCard` (renders every Ask outcome and always shows the disclaimer), `OfficeContact` and more.	nothing
`src/pages/admin/` Mockup	`AdminOverview` with sample numbers from `sampleData.ts` and a "Sample data" badge. Charts in `charts/` are plain SVG with no chart library.	nothing yet

How the modules relate

Dependencies point one way: pages → data access → HTTP client on the frontend, and routers → auth and schemas → models → database on the backend. A few rules hold this together:

- **The API shape lives in three places that change together:** `docs/api.md` (for people), `backend/app/schemas/` (FastAPI checks responses against it) and `frontend/src/api/types.ts` (the frontend build fails if code uses a field that isn't there).
- **Routes never read cookies or identity headers.** They depend on `get_current_user` or `require_role`, so replacing the mock login with UNC SSO doesn't touch them.

- **Pages never call `fetch`.** Researcher pages go through `researcherApi.ts` (one function per endpoint), which uses `api/client.ts`. Swapping fake data for a real endpoint changes one function.
- **Roles come from Nora's database, never the identity provider.** The frontend hides admin UI, but only the backend's `require_role()` enforces access.
- **Each model module is imported in `app/models/__init__.py`**, which is how Alembic finds tables for autogenerated migrations.

The README

Copied verbatim from README.md on `dev` (commit `60fa742`). It explains how to build and run the system from the repository.

UNC Research Policy Navigator

A web app that helps UNC researchers find and understand research policies. Researchers can browse topics, search the policy library, or ask a plain-language question and get an answer with citations. UNC Research staff use an admin portal to manage content and see analytics.

Built for COMP 523 with the UNC Office of the Vice Chancellor for Research (Research Compliance Services).

Design docs, the roadmap and the decision log are in docs/.

Stack

Layer	Tech
Frontend	React + TypeScript, built with Vite
Backend	FastAPI (Python 3.12), SQLAlchemy, Alembic
Database	PostgreSQL 17 + pgvector
Deployment	Docker images, targeting Carolina CloudApps (OpenShift)

Running locally

Option A: everything in Docker (easiest)

```
docker compose up --build
```

- App: http://localhost:5173
- API docs: http://localhost:8000/api/docs
- Postgres: `localhost:5432`, user/password `postgres` / `postgres`, database `policy_navigator`

Migrations and dummy seed data are applied on startup. Sign in with the mock login as `researcher1` or `admin1` (no password).

Code changes hot-reload in both the frontend and backend. Stop with `Ctrl+C`. To wipe the database, run `docker compose down -v`.

Option B: backend and frontend on your machine

Start just the database with `docker compose up db`, then:

```
# Backend (Python venv lives at the repo root)
python3 -m venv .venv && source .venv/bin/activate
pip install -e "./backend[dev]"
cd backend
alembic upgrade head
python -m app.seed                  # dummy users and data
uvicorn app.main:app --reload       # http://localhost:8000

# Frontend (separate terminal)
cd frontend
npm install
npm run dev                         # http://localhost:5173
```

How the frontend reaches the API

The frontend always calls relative paths like `/api/health`; it never knows the backend's URL. Something in front of it forwards `/api/*` to FastAPI:

Environment	Who serves the frontend	Who forwards `/api/*`
Local dev	Vite dev server (5173)	Vite's `server.proxy` (vite.config.ts)
Production	nginx serving the built static files (8080)	nginx `proxy_pass` (nginx.conf.template)

Because the browser only talks to one origin, no CORS setup is needed.

Tests and lint

```
cd backend && pytest                            # 100% coverage required
cd backend && ruff check . && ruff format --check .
cd frontend && npm run lint && npm run build
```

CI runs all of these, plus migrations and Docker builds, on every PR.

Git workflow

```
feature/* ──> dev (default) ──> qa (testing) ──> main (production)
```

- Branch off `dev` using `feature/`, `fix/`, `chore/`, `docs/`, `refactor/` or `test/` plus a lowercase name, e.g. `feature/ask-endpoint`.
- Open a PR into `dev`. Only `dev` can merge into `qa`, and only `qa` into `main`; the **Branch rules** check enforces this.
- Commit messages have a short title line, a blank line, and a description of what and why, unless the change is trivial.

Database migrations

```
cd backend
alembic revision --autogenerate -m "add users table"   # after changing models
alembic upgrade head
```

Review autogenerated migrations before committing them.

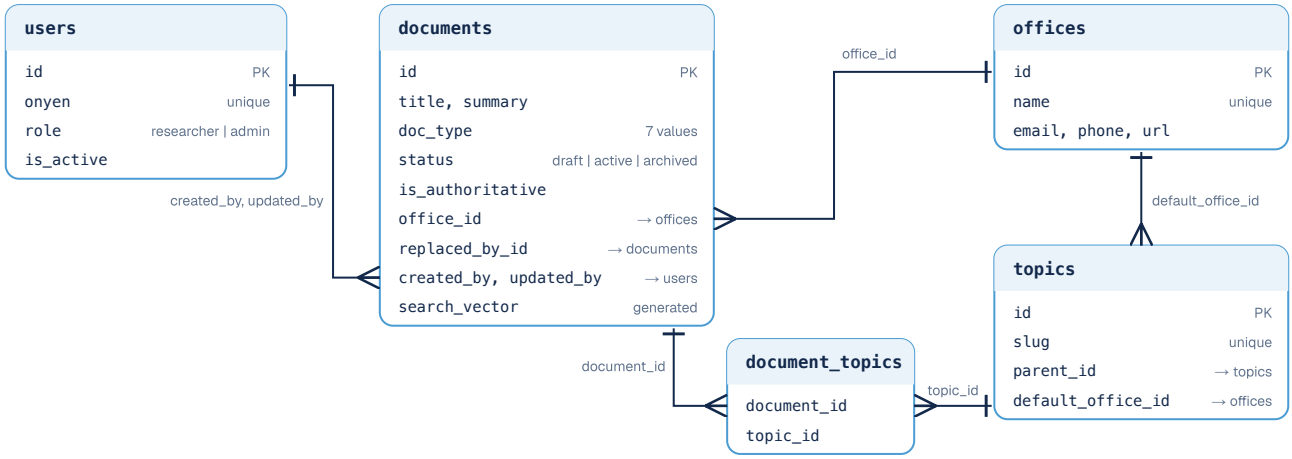
3. Detailed Data Definitions

Nora keeps all of its data in one PostgreSQL database. Five tables exist today; the rest are designed and listed at the end. Seed data comes from one JSON file, and configuration from environment variables.

Conventions

- The schema is defined in `backend/app/models/` and created by the migrations in `backend/alembic/versions/`. If this page and the code disagree, the code is right.
- Primary keys are `id bigint generated always as identity`: the database assigns them and they can't be set by hand.
- Constraint names follow a convention set in `app/db.py`, such as `pk_users`, `uq_users_onyen` and `ck_users_role_valid`, so later migrations can find them by name.
- Enumerations are stored as text with a CHECK constraint rather than Postgres enum types, which are harder to change in migrations.
- Timestamps are `timestamptz` (time zone aware). `created_at` defaults to `now()`.
- Content is archived (`status = 'archived'`), never deleted by normal app flows.

Tables and relationships



Lines run from a foreign key (crow's foot: many) to the row it points at (bar: one). A document has one office and many topics; a topic has one default office and many documents.

`users`

Everyone who has signed in, and their role. Created by migration `6ddc0a734e48`.

COLUMN	TYPE	NULL	DEFAULT / CONSTRAINTS	USE
id	`bigint`	no	PK, identity	The id stored in the session cookie.
onyen	`varchar(64)`	no	unique	The person's UNC identity. Always lowercase and matching `[a-z0-9._-]{1,64}` (checked by the API, not the database). Comes from SSO later; the mock login supplies it now.
display_name	`varchar(200)`	yes		Shown in the header. From SSO attributes, updated on login.
email	`varchar(320)`	yes		Contact address, from SSO attributes.
role	`varchar(20)`	no	default `'researcher'`; CHECK in ('researcher', 'admin')	What the user may do. Re-read on every request.
is_active	`boolean`	no	default `true`	A disabled user (false) gets 401 on every request and 403 at login.
created_at	`timestamptz`	no	default `now()`	First sign-in.
last_login_at	`timestamptz`	yes		Set by `record_login()` on every sign-in; feeds user activity analytics.

offices

UNC offices that own documents and receive referrals. Created by migration `6de3ee5b4d8c`.

COLUMN	TYPE	NULL	DEFAULT / CONSTRAINTS	USE
id	`bigint`	no	PK, identity	
name	`varchar(200)`	no	unique	Display name. The seed script matches offices by name.
description	`text`	yes		Optional notes about the office.
email	`varchar(320)`	yes		General office contact shown in referrals. Never an individual's address.
phone	`varchar(50)`	yes		General office phone, shown in referrals.

COLUMN	TYPE	NULL	DEFAULT / CONSTRAINTS	USE
url	varchar(2000)	yes		The office's website.

topics

The shared taxonomy for browsing documents and, later, classifying questions, so analytics line up with content. Created by migration `6de3ee5b4d8c`.

COLUMN	TYPE	NULL	DEFAULT / CONSTRAINTS	USE
id	bigint	no	PK, identity	
slug	varchar(100)	no	unique	URL identifier, such as `human-subjects-research` in `/browse/:slug` and `?topic=`.
name	varchar(200)	no		Display name, such as "Human Research & IRB".
description	text	no	default `''`	One line shown on the topic card.
parent_id	bigint	yes	FK → topics.id	For optional subtopics. Null for top-level topics; all 9 seeded topics are top-level while the client revises the subtopic framework.
default_office_id	bigint	yes	FK → offices.id	The office shown on the topic page and used for referrals in that topic.
sort_order	integer	no	default `0`	Display order (ascending), then name.
is_active	boolean	no	default `true`	Inactive topics are hidden from the API.

documents

One row per source document: a policy, SOP, guidance page, FAQ, web page or job aid. Created by migration `6de3ee5b4d8c`.

COLUMN	TYPE	NULL	DEFAULT / CONSTRAINTS	USE
id	bigint	no	PK, identity	Used in `/documents/:id`.
title	varchar(500)	no		Display title. Weighted highest in search.
doc_type	varchar(20)	no	CHECK in ('policy', 'sop', 'guidance', 'faq', 'webpage', 'job_aid', 'other')	Kind of document, shown as a badge.
citation	varchar(200)	yes		Official number or reference, such as "OHRE SOP 1301".
source_url	varchar(2000)	yes		The official page users are sent to. The seed script uses it as the document's identity.
summary	text	yes		Short description for browse and search results. Weighted second in search.
content_text	text	no	default `''`	Full extracted text. Weighted lowest in search; the input to RAG chunking.
content_hash	varchar(64)	yes		SHA-256 of `content_text`, in hex. Detects changed or duplicate uploads.
office_id	bigint	yes	FK → offices.id	The owning office.
status	varchar(20)	no	default `'draft'`; CHECK in ('draft', 'active', 'archived')	Lifecycle. Only `active` documents are returned by the API.
is_authoritative	boolean	no	default `false`	Only documents that are both authoritative and active will be used for answers.
replaced_by_id	bigint	yes	FK → documents.id	Points to the newer document that supersedes this one.
effective_date	date	yes		When the policy took effect.
last_reviewed_at	date	yes		Last content review, shown to users.

COLUMN	TYPE	NULL	DEFAULT / CONSTRAINTS	USE
review_due_at	date	yes		Next review date, for future "stale content" alerts.
created_by, updated_by	bigint	yes	FK → users.id	The admin who created or last changed the row (admin portal, planned). Null for seeded rows.
created_at, updated_at	timestamptz	no	default `now()`	`updated_at` is reset on every ORM update.
search_vector	tsvector	no	generated, stored; GIN index	Computed by Postgres from title (weight A), summary (B) and content_text (C) with the English dictionary. Never written directly.

document_topics

Links documents to topics; a document can belong to several. Created by migration `6de3ee5b4d8c`.

COLUMN	TYPE	NULL	DEFAULT / CONSTRAINTS	USE
document_id	bigint	no	PK part; FK → documents.id, ON DELETE CASCADE	
topic_id	bigint	no	PK part; FK → topics.id, ON DELETE CASCADE; indexed	The index makes "documents in this topic" fast.

Rules between fields

These relationships between fields aren't visible from the column types. Where the database doesn't enforce a rule, the application must.

- A document appears in the API only when `status = 'active'`. Draft and archived documents are invisible to researchers, and a topic's `document_count` counts only active documents.
- A document will be used to answer questions only when `status = 'active'` **and** `is_authoritative = true`.
- `content_hash` must equal the SHA-256 of `content_text`; set it whenever `content_text` changes.
- `search_vector` is derived from `title`, `summary` and `content_text`, and Postgres recomputes it whenever they change.

- `replaced_by_id` should point to a different document, and the replaced document is normally archived. The database doesn't enforce either rule yet.
- `topics.parent_id` should not form a cycle (a topic can't be its own ancestor). Not enforced; no subtopics exist yet.
- `users.onyen` is stored trimmed and lowercase. The API normalizes it before every lookup, so `JDoe` and `jdoe` are the same user.
- A topic's `default_office_id` is the referral office for questions in that topic. A document's `office_id` is the office that owns that document. They usually match but don't have to.
- `review_due_at` should be after `last_reviewed_at` when both are set. Not enforced.

Seed data file: research_policies.json

backend/app/seed/data/research_policies.json is a JSON array with one object per policy. Its 16 entries are real public policies from the UNC Policy Knowledge Base. The seed script also defines 8 offices, 9 topics and the 2 demo users (`researcher1` , `admin1`) in Python.

FIELD	FORMAT AND RULES
`title, doc_type, citation, summary`	Copied into the matching `documents` columns. `doc_type` must be one of the allowed values.
`source_url`	Required and unique. It's the key: re-running the seed updates the document with this URL instead of adding a copy.
`office`	A key of `SEED_OFFICES` in `app/seed/content.py` : `ovcr` , `ohre` , `oct` , `coi` , `osp` , `oacu` , `export` or `opa` .
`topics`	A list of slugs from `SEED_TOPICS` in the same file.
`content_text`	The full text. The seed sets `content_hash` , `status = 'active'` and `is_authoritative = true` .

Session cookie

`nora_session` is the only data stored in the browser. Starlette signs it with `SESSION_SECRET` and it holds one value, `user_id` (the `users.id`). It's HTTP-only (JavaScript can't read it), SameSite=Lax, expires after `SESSION_MAX_AGE_SECONDS` , and is HTTPS-only in production. Logging out clears it.

Configuration (environment variables)

VARIABLE	DEFAULT	USE
DATABASE_URL	local Postgres on 5432	SQLAlchemy connection string, `postgresql+psycopg://user:pass@host:port/db`.
ENVIRONMENT	local	`production` turns on HTTPS-only cookies and the startup safety checks.
AUTH_PROVIDER	mock	Which sign-in provider to use. Only `mock` exists; `shibboleth` is planned.
SESSION_SECRET	a development value	Signs the session cookie. Anyone who knows it can forge a login. Required in production (`openssl rand -hex 32`).
SESSION_MAX_AGE_SECONDS	28800 (8 hours)	How long a sign-in lasts.
CORS_ORIGINS	empty	Only needed if the frontend is served from a different origin than the API.
POSTGRES_USER, _PASSWORD, _DB	postgres, postgres, policy_navigator	Docker Compose only: credentials for the local database container.
API_PROXY_TARGET	http://localhost:8000	Frontend development: where Vite forwards `/api`.
BACKEND_URL	http://backend:8000	Frontend production image: where nginx forwards `/api`.

Planned tables

Designed in docs/database.md, which has their full column lists. They'll be added through migrations as the features that need them are built.

TABLE	USE	
document_chunks	The RAG units: section-sized pieces of each document with their heading, token count, embedding (`vector(384)` for the planned bge-small model, HNSW index) and a `tsvector`. Regenerated when a document changes.	Planned
questions	One row per Ask: the question, classified topic, outcome (`answered`, `referred`, `no_answer`, `error`), confidence, the answer text, referral office, model, prompt version and latency.	Planned

TABLE	USE	
`question_citations`	Which chunks were retrieved and which the answer cited, with rank and score. Drives "most cited resources".	Planned
`feedback`	Helpful or not, plus an optional comment, one per user per question.	Planned
`faqs, faq_documents`	Admin-written FAQs and the documents they cite. Never auto-published.	Planned
`saved_items`	A researcher's saved documents or answers (exactly one of the two per row).	Planned
`usage_events`	Logins, topic browses, searches, document views and source link clicks, for activity analytics.	Planned

4. Design Rationale

The decisions that shaped Nora, why we made them, and what we chose not to do. The dated, running record is docs/decisions.md; our meeting notes show where many of them came from.

Design decisions

4.1 A three-tier web app behind one origin

Decision	A React single-page app, a FastAPI JSON API and PostgreSQL. A proxy (Vite in development, nginx in production) serves the app and forwards `/api/*`, so the browser sees a single origin.
Why	Each tier can be built and tested on its own, and the same images run locally and on CloudApps. One origin means no CORS configuration, and cookies just work.
Not chosen	A server-rendered framework, which would mix UI and API code that two pairs of teammates needed to build in parallel.

4.2 Python and FastAPI for the backend

Decision	FastAPI with SQLAlchemy 2 and Alembic, on Python 3.12.
Why	The RAG work (embeddings, rerankers, LLM clients) lives in the Python ecosystem. FastAPI validates requests and responses with Pydantic and generates interactive API docs at `/api/docs`.

4.3 One database: PostgreSQL with pgvector

Decision	PostgreSQL 17 stores users, content, embeddings and analytics, using pgvector for vector search and built-in full-text search for keywords.
Why	Under 200 documents and low traffic don't justify a second system. Keeping vectors next to the documents makes filtering (active, authoritative, topic) a normal SQL query.
Not chosen	A separate vector database (Pinecone, Chroma), and Supabase as anything more than hosted Postgres.

4.4 Authentication separated from authorization

Decision	A provider (mock now, Shibboleth later) establishes who the person is. Nora's own `users.role` decides what they may do. Routes depend only on `get_current_user` and `require_role`.
Why	SSO access was pending when we started. Building against an interface let us finish everything else, and switching to UNC SSO is one new provider plus `AUTH_PROVIDER=shibboleth`.

4.5 A signed cookie holding only the user id

Decision	Starlette's `SessionMiddleware` signs a cookie containing just the user id. The user and role are re-read from the database on every request.
Why	Disabling a user or changing their role takes effect immediately, there's no token handling in the frontend, and no sessions table to maintain.
Not chosen	JWTs stored in the browser, and a server-side sessions table (unless we later need to revoke sessions).

4.6 A single role column

Decision	`users.role` is `researcher` or `admin`. The admin list is simply the users whose role is `admin`.
Why	There are two roles and nobody needs both, so `roles`, `user_roles` and `permissions` tables would add joins without any benefit.

4.7 Documents, chunks, topics and offices as separate tables

Decision	`documents` are what admins manage and users see; `document_chunks` (planned) are what RAG retrieves. Topics are a shared taxonomy with many-to-many links, and offices are their own table.
Why	Admins edit whole documents while retrieval works on sections. Questions and documents sharing one taxonomy makes topic analytics line up with content, and both referrals and ownership need offices.

4.8 Weighted full-text search in the database

Decision	A generated, stored `tsvector` with a GIN index, weighting title over summary over body, queried with `websearch_to_tsquery`.
Why	Fast keyword search with stemming and familiar syntax (quotes, `or`, `-word`) and no search service. The same function becomes the keyword half of hybrid retrieval for RAG.

4.9 Archive instead of delete, and lifecycle fields on documents

Decision	Documents have `status`, `is_authoritative`, `replaced_by_id` and review dates, and normal flows never hard-delete.
Why	The client requires every source to have an owner and review cycle, outdated content to be replaceable and surfaced, and nothing deleted without explicit admin confirmation.

4.10 Grounded answers or a referral, never a guess (planned RAG)

Decision	Ask retrieves passages (hybrid keyword and vector search, then a reranker). If the best score is below a threshold, Nora skips the LLM and refers the user to the topic's office. Otherwise the LLM answers only from numbered passages and cites them. Every answer shows a disclaimer.
Why	Researchers act on this guidance, so a wrong answer is worse than none. The gate also makes refusals free, since no LLM call is made. See the Ethics 1 paper's MyCity case for what happens without these safeguards.

4.11 Local AI models to start

Decision	Embeddings with `BAAI/bge-small-en-v1.5` and a local cross-encoder reranker; the LLM sits behind an `LLMProvider` interface.
Why	UNC hasn't approved any AI provider. Local models are free, need no API key, and send no researcher data off our servers. Tests use fake providers and never call real AI.

4.12 Plain React with no UI or chart framework

Decision	React Router v7 in declarative mode, plain CSS with UNC colors as variables in `theme.css`, and charts drawn in plain SVG.
Why	Few dependencies for the next team to learn or upgrade. Navy is used for text and links, and Carolina Blue only for accents, because Carolina Blue text on white fails contrast checks.

4.13 The demo sign-in page doesn't imitate UNC's login

Decision	The mock sign-in uses Nora's own theme and is labeled "Demo sign-in. UNC SSO will replace this."
Why	So the demo can never be mistaken for, or misused as, a real UNC login page.

4.14 Tests against real Postgres, with 100% backend coverage

Decision	Backend tests run against a real Postgres database, each inside a transaction that's rolled back. CI fails below 100% line coverage.
Why	Full-text search, CHECK constraints and generated columns only behave correctly in real Postgres. Rolling back keeps tests independent and fast.

4.15 Git flow enforced in CI

Decision	`feature/*` → `dev` → `qa` → `main`. A CI check rejects a PR into `qa` from anything but `dev`, into `main` from anything but `qa`, and into `dev` from a branch not named `<type>/<short-name>` (such as `feature/ask-endpoint`).
Why	`dev` is where work integrates, `qa` is what gets tested, and `main` is what's deployed.

4.16 The client's team hosts Nora

Decision	UNC Research runs the Carolina CloudApps deployment (decided Oct 27, 2026). We keep the images OpenShift-ready and every setting configurable by environment variable.
Why	The service will outlive the course. Images run as non-root under any user id and write only to `/tmp`, as OpenShift requires.

Dependencies, and why we depend on them

Exact versions are pinned in backend/pyproject.toml and frontend/package.json.

DEPENDENCY	AREA	WHY
FastAPI	Backend	Web framework: routing, dependency injection (used for sessions and auth) and automatic API docs.
uvicorn	Backend	The server that runs FastAPI, with hot reload in development.
Pydantic + pydantic-settings	Backend	Validates request and response bodies, and reads settings from environment variables.
SQLAlchemy 2	Backend	ORM and query builder; keeps queries in Python and composable, as `search_documents()` shows.
psycopg 3	Backend	The PostgreSQL driver SQLAlchemy uses.
Alembic	Backend	Versioned schema migrations, autogenerated from the models.
itsdangerous	Backend	Signs the session cookie (required by Starlette's SessionMiddleware).
pytest, pytest-cov, httpx	Backend (dev)	Tests, the coverage check, and FastAPI's test client.
Ruff	Backend (dev)	Linting and formatting in one fast tool.
React 19, react-dom	Frontend	UI library; the most widely known choice for a future team.
React Router 7	Frontend	Client-side routes and the nested auth and role guards.
Vite + @vitejs/plugin-react	Frontend (dev)	Dev server with hot reload and the `/api` proxy, and the production build.
TypeScript	Frontend (dev)	Type checks against `types.ts` catch API mismatches at build time.
oxlint	Frontend (dev)	Fast linting.
PostgreSQL 17 + pgvector	Infrastructure	The database, as the `pgvector/pgvector:pg17` image locally.
Docker and Docker Compose	Infrastructure	Identical environments for every developer, CI and production.

DEPENDENCY	AREA	WHY
nginx-unprivileged	Infrastructure	Production web server that runs as non-root, as OpenShift requires.
GitHub Actions	Infrastructure	CI: lint, tests, migrations, image builds and branch rules on every PR.
UNC Shibboleth SSO	External (planned)	Production sign-in with Onyens. Requested from UNC ITS.
Carolina CloudApps	External	Production hosting, run by the client's team.
Embedding, reranker and LLM models	External (planned)	RAG. Local models first; the LLM provider is pending UNC approval.

What we chose not to do

NOT DONE	WHY
No Supabase Auth or other third-party login	UNC SSO is the only identity source; roles stay in our database.
No roles or permissions tables	Two roles, and nobody needs both.
No separate vector database or search service	Postgres handles both at our scale.
No live sync with the Policy Knowledge Base	Out of scope. Admins choose and load documents, which keeps unapproved content out.
No stored analytics tables	Dashboard numbers will be SQL queries over the event tables. At this scale precomputed stats aren't needed.
No sessions table	The signed cookie is enough until we need server-side revocation.
No Postgres enum types	Text with a CHECK constraint is easier to change in a migration.
No CSS framework or chart library	Fewer dependencies; the admin charts are small enough to draw in SVG.
No auto-published FAQs	Admins review and write every FAQ, as the client requires.
No integration with IRB, COI or other enterprise systems, and no confidential records	Out of scope; Nora uses only curated public or client-approved documents.

NOT DONE	WHY
No MFA of our own, and English only	UNC SSO provides MFA; multiple languages are out of scope this semester.
No compliance determinations	Nora gives informational guidance with sources and refers anything else to the responsible office.

Open questions

Still waiting on answers from the client, the Link or UNC ITS. Each one affects the code where noted.

- **Privacy:** may researchers' questions and Onyens be stored outside UNC infrastructure, and should questions be stored with the user id or anonymized? (client, ITS)
- **AI providers:** which LLM and embedding providers UNC approves. This fixes the embedding size in `document_chunks`. (client, ITS)
- **SSO:** the integration pattern on CloudApps and which attributes arrive (Onyen, name, email). A reply to our request has come in and needs recording in `docs/auth.md`. (ITS)
- **Hosting details:** Postgres with pgvector, outbound API access, image registry, hostname and secrets on CloudApps. (client's team)
- **Content:** which documents are approved to load, which office handles which topic, and the exact disclaimer and referral criteria. (the Link)
- **Subtopics:** the client is still revising the subtopic framework; `topics.parent_id` is ready for it. (client)

5. Guide for the Next Developer

Step-by-step recipes for the tasks a new developer is most likely to be handed, and the mistakes that are easy to make.

Run Nora on your machine

1. Install Docker, clone the repo, and run `docker compose up --build` from the repo root.
2. Open http://localhost:5173 and sign in as `researcher1` or `admin1`. API docs are at http://localhost:8000/api/docs.
3. Both servers hot-reload. `docker compose down -v` wipes the database; the next start re-creates and re-seeds it.

Add an API endpoint

1. Write the request and response shapes in `docs/api.md` first.
2. Add matching Pydantic models in `backend/app/schemas/` and TypeScript types in `frontend/src/api/types.ts`.
3. Add the route to a router in `backend/app/api/` (or a new router, included in `create_app()` in `main.py`). Depend on `get_current_user`, or on `require_role(Role.ADMIN)` for admin routes.
4. Write tests in `backend/tests/` using the `client` and `make_user` fixtures and the `login` helper. Coverage must stay at 100%.
5. Add a function to `researcherApi.ts` (or an admin equivalent) and call it from the page.

Add or change a table

1. Edit or add a model in `backend/app/models/`; import any new module in `app/models/__init__.py`. Name CHECK constraints (`name="role_valid"`).
2. From `backend/`, run `alembic revision --autogenerate -m "what changed"`, then read the generated file and fix anything Alembic missed (for example, generated columns or extensions).
3. Run `alembic upgrade head`, update the seed script if needed, and update `docs/database.md` and this document.
4. If two branches each add a migration, set one's `down_revision` to the other's revision so Alembic doesn't end up with two heads.

Add a frontend page

1. Create the component in `frontend/src/pages/researcher/` or `pages/admin/`.
2. Add its route in `App.tsx`, inside the right `AppLayout` (and under `RequireRole` for admin pages).
3. Give its item in `layout/navItems.ts` a `to` so it stops showing as "Soon".
4. Use the variables in `theme.css` for colors and spacing; check it at phone width (the layout switches below 48rem).

Add policies to the seed data

1. Append an entry to `backend/app/seed/data/research_policies.json` using the fields in the Data Definitions section.
2. Run `python -m app.seed` (or restart Docker Compose). The seed is idempotent, so re-running it updates existing rows instead of duplicating them.

Build Ask (RAG)

1. Read `docs/rag.md`: the stages, retrieval design, gate and evaluation plan.
2. Implement `POST /api/ask` with the response shape already agreed in `docs/api.md`. Put AI calls behind `EmbeddingProvider` and `LLMProvider`, with fakes for tests.
3. In `researcherApi.ts`, replace the body of `ask()` with the `apiPost` call in its comment, then delete `fakeData.ts`. `AnswerCard` already renders every outcome.

Switch to UNC SSO

1. Add a Shibboleth provider that reads the identity headers set by the SSO proxy (trusting them only from the proxy), then calls `record_login()` and sets the session.
2. Allow `shibboleth` in `Settings.auth_provider` and set `AUTH_PROVIDER=shibboleth`. The mock login router then isn't mounted.
3. Point logout at the SSO logout URL. Routes need no changes. See `docs/auth.md`.

Things that will trip you up

- Production refuses to start if `AUTH_PROVIDER=mock` or `SESSION_SECRET` is the development default. That's intentional.
- CI fails if backend coverage drops below 100%, if Ruff finds issues, if migrations don't apply, or if either Docker image fails to build.
- Changing an API shape means changing `docs/api.md`, the Pydantic schema and `types.ts` in the same PR.
- Carolina Blue (`#4B9CD3`) is too light for text on white. Use navy for text and links.
- The seed policies' `content_text` has its headings flattened, so RAG ingestion must re-extract the HTML to chunk by section.
- Each reranker scores on its own scale, so re-tune the confidence gate's threshold whenever the reranker changes.
- The project's design records in `docs/` are kept current with each change. When a decision is made, add a dated entry to `docs/decisions.md`.

6. Related Documents

This is the one document meant to outlive the semester, so it points to everything else.

Specifications (D1) →

User stories and the functional, non-functional and interface requirements (the F, NF and I numbers referenced in code and docs).

Test Plan (D3) →

What we test and how: unit, integration, system and acceptance testing.

User Manual (D4) →

Guides for researchers and administrators. Coming later this semester.

Code (D5) →

The final code deliverable.

Architecture Diagram (M7) →

The first, simpler architecture diagram, including the planned production setup.

Platform Selection (M6) →

The alternatives we compared for each part of the stack.

System Metaphor (M5) →

Nora as a reference library: a quick mental model.

Client Handoff Plan (M8) →

How the code, hosting and accounts pass to UNC Research.

Design records in the repository

The `docs/` folder holds the detailed, continuously updated records this document summarizes. When they disagree with this document, they (and the code) are newer.

FILE	WHAT IT COVERS
`docs/product.md`	Clients, users, product rules and the admin dashboard requirements
`docs/api.md`	Every endpoint's request and response
`docs/database.md`	The full schema, including the planned tables
`docs/auth.md`	Mock login, UNC SSO plans and the admin list
`docs/frontend.md`	Routes, layout, navigation and styling rules
`docs/rag.md`	The Ask pipeline design, models and evaluation

FILE	WHAT IT COVERS
`docs/deployment.md`	Environments and CloudApps requirements
`docs/roadmap.md`	Milestones and what's built
`docs/decisions.md`	Dated decision log and open questions

7. Glossary

The domain and technical terms used in this document and the code.

Nora
Nora the Policy Navigator, this application. The repository's older name, UNC Research Policy Navigator, still appears in places.

Researcher
Anyone at UNC doing research (faculty, staff, students, coordinators) who uses Nora to find policy. The default role.

Admin
UNC Research staff who manage Nora's content and see its analytics. The only other role.

Onyen
A UNC user ID, such as `jdoe`. It's the identity Nora stores for every user (`users.onyen`).

SSO / Shibboleth
UNC's single sign-on, where people log in with their Onyen. Shibboleth is the software behind it. Nora will use it in production; until UNC grants access, a **mock login** stands in for it.

Carolina CloudApps (OKD / OpenShift)
UNC's container hosting platform, a Kubernetes distribution. Production Nora will run there; the client's team operates it.

OVCR / Research Compliance Services
The Office of the Vice Chancellor for Research and its compliance group: our client (Jeanne Lovmo and Quinton Johnson).

The Link
The Research Policy Navigator Link, the client's working group. It owns requirements, content rules, test scenarios and user acceptance testing.

UNC Policy Knowledge Base
policies.unc.edu, where UNC publishes its policies. Its Research section (9 categories, under 200 documents) is Nora's content source. Also called the Research Policy Repository.

Office
A UNC office responsible for an area, such as the Office of Human Research Ethics. Offices own documents and are where Nora refers people.

IRB, COI, IACUC
Common policy areas: the Institutional Review Board (human subjects research), conflict of interest, and the Institutional Animal Care and Use Committee.

RAG
Retrieval-augmented generation: find the relevant policy passages first, then have a large language model (LLM) write an answer using only those passages, citing them.

Chunk / embedding

A chunk is a section-sized piece of a document, the unit RAG retrieves and cites. An embedding is a list of numbers representing a text's meaning, so similar texts have nearby embeddings.

pgvector

A PostgreSQL extension that stores embeddings and finds the nearest ones. Enabled now; used once RAG is built.

Full-text search / tsvector

PostgreSQL's built-in keyword search. A `tsvector` is a pre-processed list of a document's words, which makes searching fast.

Migration

A versioned script (we use Alembic) that changes the database schema, so every database can be brought to the same structure.

Seed data

Fake users and real public policies loaded by a script, so every developer's local database looks the same.

SPA

Single-page application: the browser loads the React app once, and it then fetches data from the API as JSON.